

```
akshi@akshita:~$ mkdir ~/parser_lab
cd ~/parser_lab
akshi@akshita:~/parser_lab$ nano tokenizer_parser.c
akshi@akshita:~/parser_lab$ gcc tokenizer_parser.c -o tokenizer_parser
akshi@akshita:~/parser_lab$ ./tokenizer_parser
=== Simple Tokenizer + Parser ===
Enter a command line (or an empty line to test error handling):
> ls -l /home

--- Stage 1: Tokenizing ---
Token count: 3
Token[0] = "ls"
Token[1] = "-l"
Token[2] = "/home"

--- Stage 2: Validating token stream ---
Token stream is valid.

--- Stage 3: Parsing into execution structure ---
[PARSE RESULT] Valid command structure:
  argc = 3
  Command name (argv[0]) = "ls"
  Arguments:
    argv[1] = "-l"
    argv[2] = "/home"
    argv[3] = NULL (terminator, required by exec())
akshi@akshita:~/parser_lab$ ./tokenizer_parser
=== Simple Tokenizer + Parser ===
Enter a command line (or an empty line to test error handling):
> ls -l /home

--- Stage 1: Tokenizing ---
Token count: 3
Token[0] = "ls"
Token[1] = "-l"
Token[2] = "/home"

--- Stage 2: Validating token stream ---
Token stream is valid.

--- Stage 3: Parsing into execution structure ---
```

```

--- Stage 3: Parsing into execution structure ---
[PARSE RESULT] Valid command structure:
    argc = 3
    Command name (argv[0]) = "ls"
    Arguments:
        argv[1] = "-l"
        argv[2] = "/home"
        argv[3] = NULL (terminator, required by exec())
akshi@akshita:~/parser_lab$ ./tokenizer_parser
=== Simple Tokenizer + Parser ===
Enter a command line (or an empty line to test error handling):
>

--- Stage 1: Tokenizing ---
Token count: 0

--- Stage 2: Validating token stream ---
[VALIDATION ERROR] Empty command – no tokens found.

--- Stage 3: Parsing into execution structure ---
[VALIDATION ERROR] Empty command – no tokens found.
[PARSE RESULT] Invalid / empty command – nothing to execute.
akshi@akshita:~/parser_lab$ █

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

#define MAX_TOKENS 64
#define MAX_TOKEN_LEN 128
#define MAX_LINE 512

```

```

// ----- Token structure -----

```

```

typedef struct {
    char tokens[MAX_TOKENS][MAX_TOKEN_LEN];
    int count;
} TokenStream;

```

```

// ----- Parsed command structure (the "execution structure") -----

```

```

typedef struct {
    char *argv[MAX_TOKENS + 1]; // +1 for NULL terminator, like real exec() needs
    int argc;
    int valid; // 0 = invalid/empty command, 1 = valid
} Command;

```

```

// ----- Step 1: Tokenizer -----
// Splits input on whitespace (spaces, tabs), skips extra/multiple whitespace.
TokenStream tokenize(char *input) {
    TokenStream stream;
    stream.count = 0;

    // strtok splits on any of the delimiter characters, and automatically
    // collapses consecutive delimiters (so multiple spaces are handled).
    char *delimiters = " \\t\\n";
    char *token = strtok(input, delimiters);

    while (token != NULL && stream.count < MAX_TOKENS) {
        strncpy(stream.tokens[stream.count], token, MAX_TOKEN_LEN - 1);
        stream.tokens[stream.count][MAX_TOKEN_LEN - 1] = '\\0';
        stream.count++;
        token = strtok(NULL, delimiters);
    }

    return stream;
}

// ----- Step 2: Validate token stream -----
int validate_tokens(TokenStream *stream) {
    if (stream->count == 0) {
        printf("[VALIDATION ERROR] Empty command — no tokens found.\\n");
        return 0;
    }
    // check for any suspicious empty-string tokens (shouldn't happen with strtok,
    // but good defensive practice)
    for (int i = 0; i < stream->count; i++) {
        if (strlen(stream->tokens[i]) == 0) {
            printf("[VALIDATION ERROR] Empty token found at position %d.\\n", i);
            return 0;
        }
    }
    return 1;
}

// ----- Debug: print tokens -----
void debug_print_tokens(TokenStream *stream) {
    printf("Token count: %d\\n", stream->count);
    for (int i = 0; i < stream->count; i++) {
        printf(" Token[%d] = \"%s\\n", i, stream->tokens[i]);
    }
}

```

```

    }
}

// ----- Step 3: Parser — build the execution structure -----
Command parse(TokenStream *stream) {
    Command cmd;
    cmd.argc = 0;
    cmd.valid = 0;

    if (!validate_tokens(stream)) {
        cmd.valid = 0;
        cmd.argv[0] = NULL;
        return cmd;
    }

    for (int i = 0; i < stream->count; i++) {
        cmd.argv[i] = stream->tokens[i];
        cmd.argc++;
    }
    cmd.argv[cmd.argc] = NULL; // exec()-style argv arrays must end with NULL
    cmd.valid = 1;

    return cmd;
}

// ----- Debug: print parsed execution structure -----
void debug_print_command(Command *cmd) {
    if (!cmd->valid) {
        printf("[PARSE RESULT] Invalid / empty command — nothing to execute.\n");
        return;
    }
    printf("[PARSE RESULT] Valid command structure:\n");
    printf("  argc = %d\n", cmd->argc);
    printf("  Command name (argv[0]) = \"%s\"\n", cmd->argv[0]);
    printf("  Arguments:\n");
    for (int i = 1; i < cmd->argc; i++) {
        printf("    argv[%d] = \"%s\"\n", i, cmd->argv[i]);
    }
    printf("  argv[%d] = NULL (terminator, required by exec())\n", cmd->argc);
}

int main() {
    char line[MAX_LINE];

```

```
printf("=== Simple Tokenizer + Parser ===\n");
printf("Enter a command line (or an empty line to test error handling):\n> ");

if (fgets(line, sizeof(line), stdin) == NULL) {
    printf("No input received.\n");
    return 1;
}

printf("\n--- Stage 1: Tokenizing ---\n");
TokenStream stream = tokenize(line);
debug_print_tokens(&stream);

printf("\n--- Stage 2: Validating token stream ---\n");
if (validate_tokens(&stream)) {
    printf("Token stream is valid.\n");
}

printf("\n--- Stage 3: Parsing into execution structure ---\n");
Command cmd = parse(&stream);
debug_print_command(&cmd);

return 0;
}
```